

Speech and language technology (DA 3452)

...

Lecture 2 – July 29
Characters, Unicode and Text Normalization

Course Admin Details

Some clarifications

We will decouple this course as much as possible from the Deep Learning course

Pre-requisites

1. Linear Algebra, Calculus, Probability
2. ML-1, Corresponding Lab
3. Sound knowledge of Python

In case someone is uncomfortable with the Neural Network portions we will either have –

1. Classes (in case this is a large number of students)
2. Assigned readings (in case it is a small number of students < 5)

Attendance, Evaluation, etc

There will be no attendance maintained

If you feel ill, bored, sleepy (before or) during the class, you need not attend!

Evaluation

1. Reading/coding assignments (for 6 hours/week)
2. In class tests (typically, Friday) for each module. Includes assignments eval
3. End-semester (50%)

If there is a project, etc it will either be tested through the End-sem OR might substitute for a missed or poorly done test.

Recall : Course Organization (6 Modules)

Understand how computers can represent, understand, learn from, generate, and interact using human language.

1. Representing Language Computationally
2. Predicting Language and Linguistic Labels
3. Learning and Generating Sequences
4. Foundation Models and Adaptation
5. Knowledge-Grounding, Evaluation and Robustness
6. Speech Representation, Recognition and Synthesis

Module 1

Representing Language Computationally

Chapter 2 (J&M)

Recap : How many words in this sentence?

“They picnicked by the pool, then lay back on the grass and looked at the stars.”

16 if punctuation is excluded

18 if punctuation is counted

The answer depends on the representation and the task.

Recap : What counts as a word?

I'm

one written word? or I + am?

don't

one unit? or do + not?

New York

two words? or one named expression?

uh

removable filler? or useful speech information?

main- mainly

noise, fragment, or evidence about speech production?

A “word” is not always a universal computational unit.

Word, Morpheme and Token

Word

A familiar written or linguistic unit

Morpheme

A minimal meaning-bearing unit

Token

A computational unit produced by a tokenization procedure

cats → cat + s

worked → work + ed

carefully → care + ful + ly

I'm → I + 'm

DA3452 → may become several tokens

But even word boundaries are not obvious in all written languages!

Spaces are not universal

Whitespace is a common convention in separating words. But not universally so.

English text often gives useful cues:

where are the word boundaries?

But language data may involve:

- scripts without whitespace word marking

ฉันกำลังสอนชั้นเรียนอยู่

- morphologically rich words

पठिष्यामि

- code-switched speech and text

No obvious whitespace units

姚明进入总决赛

A single string may support multiple segmentations.

Code-switching

assignment submit pannunga

Monolingual assumptions can break unexpectedly

The Text Representation Problem – Unseen Words

Should a computer process words, morphemes, characters, or units learned from data?

Jabberwocky by Lewis Carroll

'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe:
All mimsy were the borogoves,
And the mome raths outgrabe.

"Beware the Jabberwock, my son!
The jaws that bite, the claws that catch!
Beware the Jubjub bird, and shun
The frumious Bandersnatch!"

He took his vorpal sword in hand:
Long time the manxome foe he sought—
So rested he by the Tumtum tree
And stood awhile in thought.

And, as in uffish thought he stood,
The Jabberwock, with eyes of flame,
Came whiffling through the tulgey wood,
And burbled as it came!

One, two! One, two! And through and through
The vorpal blade went snicker-snack!
He left it dead, and with its head
He went galumphing back.

"And hast thou slain the Jabberwock?
Come to my arms, my beamish boy!
O frabjous day! Callooh! Callay!"
He chortled in his joy.

The Text Representation Problem

Should a computer process words, morphemes, characters, or units learned from data?

Candidate unit	Main advantage	Main difficulty
Word	Often interpretable and meaningful	Ambiguous boundaries; unseen words
Morpheme	Meaning-bearing components	Difficult to identify consistently
Character	Reliable coverage	Often too small; long sequences
Learned subword	Compromise: coverage and length	Depends on data and method

But are even the simplest of these – **characters**, easy to represent?

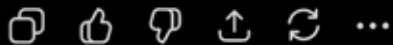
Character Representation

Unicode

A simple conversation

Hello 😊. Hum saath mein Café polaamaa?

Polaam 😊 Main physically nahi aa sakta, par virtual café company pakka—coffee tumhari, conversation meri! ☕



Notice that this single message contains :

- Ordinary “English” letters, punctuation and spaces
- An emoji
- Characters outside normal English (é)
- Mix of transliterated (written in “English”) Tamil and Hindi

Note : The LLM works not with our text directly, but only numbers

What happens behind the scenes of converting the input text into numbers for the model?

The Pipeline: From Keyboard to LLM

[Your Input]



1. Web/App Interface
captures the message



2. Unicode Text
e.g., "Hello 😊" or "Café"



3. Tokenizer
converts text into token IDs



4. Embedding Layer
converts token IDs into learned vectors



5. Transformer / LLM
processes the vectors and predicts output tokens

Key idea

The LLM does not directly receive the visible letters, emoji, and punctuation that we see.

Before the network

Unicode text → token IDs

Neural model begins

embedding layer + transformer

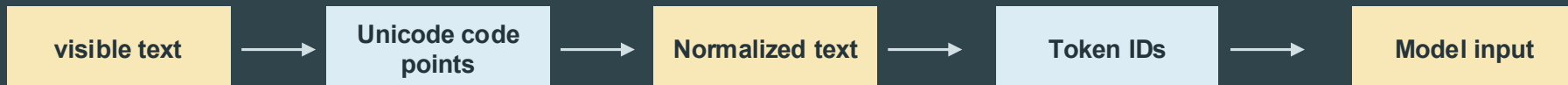
Let us see how written characters are represented computationally before tokenization.

What does a computer “see” in a character?

- Languages use many different writing systems and characters
- Modern text often mixes scripts, languages, symbols, and emoji

A · é · தமிழ் · हिंदी · 中 · 😄

The character transformation pipeline



What is Unicode?

ASCII — An Early Character Encoding

- ASCII – American Standard Code for Information Interchange
 - Developed for English-language computing
 - Its assignments are retained in Unicode’s Basic Latin block
- ASCII is a 7-bit encoding (commonly store in one 8-bit byte)
 - 128 possible codes

Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	
<	3C	60	@	40	64	...	\	5C	92	`	60	96
=	3D	61	A	41	65	...	[	5D	93	a	61	97
>	3E	62	B	42	66	...	^	5E	94	b	62	98
?	3F	63	C	43	67	...	_	5F	95	c	63	99

Figure 2.4 Some selected ASCII codes for some English letters, with the codes shown both in hexadecimal and decimal.

But ASCII cannot represent é, தமிழ், हिंदी, 中, or 😊. How can one system represent the world’s writing systems?

Unicode Standard – A common numbering for text

- **Unicode standard**

- Represents characters used in the world's writing systems
 - Includes punctuation, mathematical symbols, currency symbols, and emojis
- Unicode 17.0 has 159,801 characters

- **Each encoded character is assigned a Code Point**

- **Code Point - Numerical identifier for an abstract character**
- Written in Hexadecimal. From 0 to 0x10FFFF (which is 1,114,111 decimal)
- It is traditional to represent these code points with the prefix “U+” (U+0000 to U+10FFFF)

U+0061	a	LATIN SMALL LETTER A
U+0062	b	LATIN SMALL LETTER B
U+0063	c	LATIN SMALL LETTER C
U+00F9	ù	LATIN SMALL LETTER U WITH GRAVE
U+00FA	ú	LATIN SMALL LETTER U WITH ACUTE
U+00FB	û	LATIN SMALL LETTER U WITH CIRCUMFLEX
U+00FC	ü	LATIN SMALL LETTER U WITH DIAERESIS
U+8FDB	进	
U+8FDC	远	
U+8FDD	连	
U+8FDE	连	
U+1F600	😄	GRINNING FACE
U+1F00E	🀀	MAHJONG TILE EIGHT OF CHARACTERS

Abstract character ≠ visual representation (glyph)



Next : How do we store text using code points?

Direct storage – the obvious solution

Unicode has assigned each character a number. Why not simply store that number?

Characters	h	e	l	l	o
Unicode code points	U+0068	U+0065	U+006C	U+006C	U+006F

If every code point is stored in 4 bytes:

00 00 00 68

00 00 00 65

00 00 00 6C

00 00 00 6C

00 00 00 6F

UTF-32

- Simple and uniform
- Works for all Unicode code points
- But wasteful for ordinary English text

5 characters -> 20 bytes

Can we use WhatsApp this way? Can we be more efficient?

UTF-8 : Variable Length Encoding

- Fixed-width encoding (UTF-32) is wasteful and inefficient for large systems
 - Also zeros cause backward compatibility issues with legacy software
- UTF-8 : Most common encoding standard
 - Unicode Transformation Format - 8 bits
 - Variable length encoding
 - ASCII characters: 1 byte. Higher Unicode ranges: 2–4 bytes
 - ASCII byte sequence is same in ASCII and UTF-8

UTF-32: 00 00 00 68 | 00 00 00 65 | 00 00 00 6C | 00 00 00 6C | 00 00 00 6F : 20 bytes
UTF-8: 68 | 65 | 6C | 6C | 6F : 5 bytes

Code Points		UTF-8 Encoding			
From - To	Bit Value	Byte 1	Byte 2	Byte 3	Byte 4
U+0000-U+007F	0xxxxxxx	xxxxxxxx			
U+0080-U+07FF	0000yyyy yyxxxxxx	110yyyyy	10xxxxxx		
U+0800-U+FFFF	zzzzyyyy yyxxxxxx	1110zzzz	10yyyyyy	10xxxxxx	
U+010000-U+10FFFF	000uuuuu zzzzyyyy yyxxxxxx	11110uuu	10uuzzzz	10yyyyyy	10xxxxxx

Figure 2.5 Mapping from Unicode code point to the variable length UTF-8 encoding. For a given code point in the From-To range, the bit value in column 2 is packed into 1, 2, 3, or 4 bytes. Figure adapted from Unicode 16.0 Core Spec Chapter 3 Table 3-6.

Reading Assignment!

This brings us to one final problem...